

Finals Task 3. Simple Polymorphism

Problem. Chirp and Tweet

Create a simple program to demonstrate basic polymorphism with bird sounds.

Class - Bird:

- Methods:
 - `def make_sound(self) -> None`: An abstract method that represents making a sound. It doesn't have a specific implementation in the base class `Bird`.

Class - Sparrow (extends Bird):

- Methods:
 - `def make_sound(self) -> None`: Overrides the `make_sound` method from the base class `Bird`. It prints the sound "Chirp Chirp" when called.

Class - Parrot (extends Bird):

- Methods:
 - `def make_sound(self) -> None`: Overrides the `make_sound` method from the base class `Bird`. It prints the sound "Tweet Tweet" when called.

Class - BirdCage:

- Methods:
 - `def make_bird_sounds(self, birds: List) -> None`: Accepts a list of `Bird` objects as input. Iterates through the list of birds and calls the `make_sound` method on each bird to make its sound.

Note:

- *The test cases are not outputs of your main file but of a hidden test file. Create and implement the classes instructed to test your code.*
- *Each class should be defined in its own file, with the file name following camelCase conventions (e.g., `bankAccount.py`).*

Source code:

```

1
2 from abc import ABC, abstractmethod
3
4
5 class Bird(ABC):
6     @abstractmethod
7     def make_sound(self) -> None:
8         pass
9
10
11 class Sparrow(Bird):
12     def make_sound(self) -> None:
13         print("Chirp Chirp")
14
15
16
17 class Parrot(Bird):
18     def make_sound(self) -> None:
19         print("Tweet Tweet")
20
21
22
23 class BirdCage:
24     def make_bird_sounds(self, birds: list) -> None:
25         for bird in birds:
26             bird.make_sound()
27
28
29 def run_tests():
30     print("Test case 1: Sparrow.make_sound()")
31     Sparrow().make_sound()
32     print()
33
34     print("Test case 2: Parrot.make_sound()")
35     Parrot().make_sound()
36     print()
37
38     print("Test case 3: Polymorphism using Bird type")
39     bird1: Bird = Sparrow()
40     bird2: Bird = Parrot()
41     bird1.make_sound()
42     bird2.make_sound()
43     print()
44
45     print("Test case 4: Bird is abstract")
46     print("Bird class is abstract and cannot be instantiated.")
47     print()
48
49     print("Test case 5: BirdCage sounds")
50     cage = BirdCage()
51     cage.make_bird_sounds([Sparrow(), Parrot()])
52     print()
53
54
55 if __name__ == "__main__":
56     run_tests()
57

```

Sample output:

Output

Test case 1: Sparrow.make_sound()

Chirp Chirp

Test case 2: Parrot.make_sound()

Tweet Tweet

Test case 3: Polymorphism using Bird type

Chirp Chirp

Tweet Tweet

Test case 4: Bird is abstract

Bird class is abstract and cannot be instantiated.

Test case 5: BirdCage sounds

Chirp Chirp

Tweet Tweet